

Procedural Generation of a Coherent Infinite 2D Landscape

José Martins
University of Porto
Porto, Portugal
joseapmartins2003@gmail.com

Francisco Cardoso
University of Porto
Porto, Portugal
franciscocardoso.3003@gmail.com

Cedric Hartz
University of Paderborn
Bad Salzuflen, Germany
cedric.hartz@gmx.de

Abstract—This paper presents a procedural generation framework for an infinite, real-time 2D top-down world that achieves large-scale environmental coherence through physically inspired simulations. Although rendered in two dimensions, the system relies on continuous altitude maps to drive thermodynamics, hydrology, and biome formation, enabling effects such as snow-capped mountains, rain shadows, and river-based oasis ecosystems. To support infinite real-time generation under strict locality and performance constraints, all simulations are fully deterministic and operate at the chunk level. Biomes are assigned probabilistically using a Whittaker-based classification, producing smooth ecological transitions rather than discrete boundaries.

Index Terms—Procedural Content Generation, Infinite Worlds, Biome Generation, Environmental Simulation, Chunk-Based Generation, Climate Modeling, Terrain Generation.

I. INTRODUCTION

Many video games are centered around expansive worlds where players can fight, complete quests, explore, and build structures. As a result, game developers must create large game maps. This task is not easy, especially for small studios and teams, and requires significant time and financial resources. Consequently, the demand for Procedural Content Generation (PCG) is increasing.

The main challenge for PCG when focusing on terrain generation is to create a coherent and structured world that does not look purely random. Although different games adopt different approaches, they often share the same underlying goal: ensuring that the generated world remains consistent according to its own internal rules. Minecraft and Rimworld are two well-known examples of procedural terrain generation. Minecraft’s infinite world is composed of biomes that transition in a consistent and recognizable way, while RimWorld divides its world into multiple regions that may repeat biomes or patterns. These worlds do not necessarily follow real-world planetary rules, but they maintain internal consistency that makes them believable and playable.

Observing how these games balance repetition, large-scale structure, and local detail became the motivation for this paper. This project focuses on understanding how coherence can be preserved in an infinite world, valuing coherence over realism. To preserve flexibility and usability of the generation, reliable mechanisms such as continuous noise fields, physics-based approaches, and rule- and probability-based decisions were used. The constraint of generating an infinite world

required a real-time, performant generation and forced the implementation of simulations and decisions relying only on local data, instead of a full global simulation.

II. BACKGROUND

A. Procedural Content Generation

Procedural Content Generation (PCG) is the process of using computer algorithms to automatically create digital content, rather than building everything manually. Instead of hand-designing each mountain or placing every tree, designers develop rules and instructions for the computer to follow, enabling it to construct the world on its own. This approach allows for the creation of large environments that are unique each time the program is run. The primary challenge of PCG is to ensure that, despite the random generation, the resulting content appears natural and adheres to the laws of nature.

III. RELATED WORK

PCG is a widely studied topic. It has been used not only for games, such as Minecraft and RimWorld, as stated before, but also for many different purposes. Examples range from the generation of Super-Mario levels [1] to the procedural creation of buildings based on 2D footprints, where the generated architecture respects historical styles or planning constraints without the need for manual modeling [2], [3].

AutoBiomes (Chen et al., 2020) [4] is an article that presents a procedural terrain generation method for Unreal Engine 4. The main idea of the article is to generate terrain based on multiple environment maps in order to create a coherent, natural-looking world. It proposes a pipeline that eliminates the common usage of “simplex noise → heightmap → texture” and instead focuses on generating multiple environment maps that influence each other, in order to create a more complex and believable world. The pipeline proposed in AutoBiomes consists of 4 main steps:

- 1) **Rough-Based Terrain Generation:** The terrain is generated by multiple octaves of simplex noise using the ‘FastNoiseSIMD’ library.
- 2) **Climate Simulation:** Climate Simulation consists of a complex nested pipeline that computes temperature, wind, and precipitation maps.

- 3) Terrain Refinement: Creates biomes based on DEMs (Digital Elevation Models) and refines the terrain accordingly.
- 4) Asset Placement: Populates the biomes with assets that have four different meshes depending on the seasonal look.

IV. APPROACH AND GENERATION PIPELINE

This work is based on the AutoBiomes framework, which showed that coherent biome distributions can result from combining multiple environmental simulations. Unlike AutoBiomes, which targets offline generation of finite 3D terrains, the proposed approach adapts these principles to an infinite, real-time, 2D top-down world. This transition imposes stricter requirements on determinism, locality, and performance, necessitating a redesigned generation pipeline optimized for chunk-based world management.

The proposed system uses a layered, dependency-driven generation pipeline. Rather than mapping noise directly to terrain features, the world is built through a series of environment maps, each capturing a specific physical or ecological property. These maps are generated and processed in a predefined sequence.

Figure 1 presents an overview of the generation pipeline of this project, highlighting the dependencies between each stage. An arrow from temperature to water signifies that the generation of the water level uses the temperature. The pipeline begins with four synthetic noise maps for altitude, temperature, rivers, and wind. This establishes the broad structural patterns of the world. The raw noise maps are not applied directly to the world, but are instead processed by subsequent simulations, such as wind and precipitation. The second-to-last step is the placement of biomes, which follows a ruleset known as a Whittaker diagram and relies mainly on the temperature and precipitation maps from the previous simulation steps. The placement follows a probabilistic approach that enables gradual biome transitions and prevents sharp ecological boundaries.

Ultimately, the calculated biome distributions direct the placement of world objects and vegetation, ensuring that assets align with the local biome and climate. Every stage of the pipeline processes data at the chunk level and remains fully deterministic, enabling the world to be generated gradually and reliably as the player explores.

V. IMPLEMENTATION

With the design goals in mind, including real-time infinite world generation with local coherence, the implementation utilizes a chunk-based system. This approach allows for the loading of specific portions of the world as needed. Each chunk uses the previously mentioned generation pipeline to create its environment upon loading.

Given the constraints of finite memory, the need for determinism, and the requirement for real-time generation, Python has been chosen as the implementation language. Its syntax

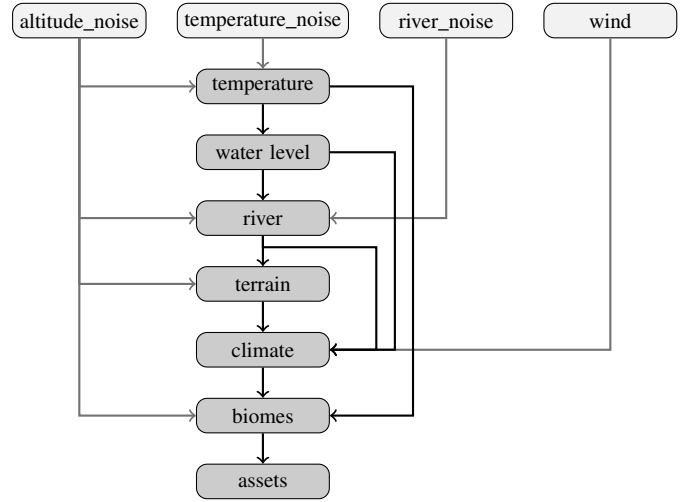


Fig. 1: Generation pipeline

is known to be easy to read, maintain, and extend for future development. Although Python generally exhibits slower performance compared to other programming languages, this limitation is addressed through the use of optimized libraries like numpy for performance-critical code.

A. Chunk System and Noise Engine

The most important aspect of generating and managing the infinite world is the chunk loading mechanism. Since it is impossible to generate and load an infinite map, the world is divided into fixed-size rectangular chunks. The generation engine loads and unloads chunks based on defined radiuses around the player.

Initially, it was assumed that each chunk required its neighboring chunks to be generated first. However, this approach led to the issue of infinite recursion when attempting to generate neighboring chunks sequentially. The proposed solution was to define two phases for the chunk generation. In the first phase, the chunk object is instantiated without creating its surrounding chunks. During the second phase, the chunk is generated. At this point, the surrounding chunk objects need to exist, but it suffices to have them created, rather than fully generated.

Figure 2 illustrates this process, showing the player as a blue dot in the middle, the generated chunks in green, the created chunks in dark red, and the non-existing chunks in black.

During the implementation, it was observed that chunks do not require their neighboring chunks to exist during the generation process. Instead, given the context of infinite environment maps, each chunk can access any values both within and beyond its own boundaries.

The noise required by the generation pipeline is implemented in a class called *NoiseEngine*. It creates noise following a similar approach to the one used in AutoBiomes, using Simplex noise together with Fractal Brownian Motion to create complex layers of Noise. The AutoBiomes paper uses the

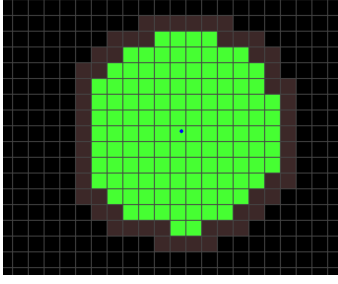


Fig. 2: Chunk generation naively takes two phases

FastNoiseSIMD library to generate noise maps. Despite this library’s good performance, it is no longer actively maintained, leading to the use of *FastNoiseLite*. This popular open-source library is a direct successor of *FastNoise*, and provides a wide range of noise algorithms and utilities for procedural content generation. It is designed for high performance and ease of use. For Python, there exists a wrapper called *pyfastnoiselite*, which was used in the project to generate various noise maps.

The project follows the generation pipeline by implementing multiple environment maps, one for each step in the pipeline. An environment map is a numpy array that maps a 2D chunk coordinate to a value. It can be thought of as a function $\mathbb{Z}^2 \rightarrow [0, 1]$, where $\mathbb{Z}^2$ is the space of (x, y) integer coordinates of the world. As an example, the water level map defines a value between 0.0 and 1.0 for each pixel. The value for a specific pixel, according to the generation pipeline, is calculated only from the temperature map.

In different places of the implementation, this project uses reshaping functions, mainly to adjust simplex noise to better fulfill its purpose. A reshaping function $[0, 1] \rightarrow [0, 1]$ is applied to an environment map per pixel. The result essentially allows the noise to be remapped to any desired shape.

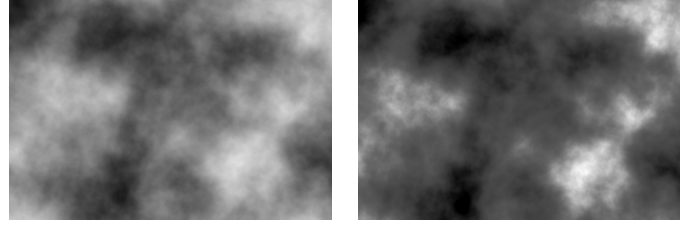
B. Terrain

The terrain map applies a reshaping function to the altitude_noise. Figure 3 shows the raw noise (a), as opposed to the reshaped noise (b). The reshaping function can be changed to fit any desired terrain shape. In this project, the high altitudes are accentuated to make the mountains more distinct and, in turn, to have more plateaus.

There is still one more step for the generation of terrain, namely, carving rivers into it. This is why the pipeline first calculates the temperature, water bodies, and rivers of the world based on the raw altitude_noise. The resulting river map is subtracted from the terrain map to carve trenches filled with water.

C. Temperature and Hydrology

The temperature map is determined by a thermodynamic model that considers both base temperature_noise and the rate at which the air cools with elevation, the adiabatic lapse rate, which results directly from altitude_noise. To implement this,



(a) Raw Noise

(b) After Reshaping

Fig. 3: Visualizing the effect of altitude reshaping. (a): Raw altitude_noise. (b): Reshaped noise.

the local temperature $T(x, y)$ is defined as a superposition of the noise map and the inverted height map:

$$T(x, y) = \text{clip}((1 + \lambda) \cdot \mathcal{N}_{temp}(x, y) - \lambda \cdot H(x, y)) \quad (1)$$

Here, $\mathcal{N}_{temp}$ represents the base temperature_noise, H is a smoothed version of altitude_noise, and λ is the cooling coefficient. This equation ensures that high-altitude terrain remains frozen regardless of the base temperature, enabling the formation of snow-capped peaks in otherwise warm zones.

At first, this project generated bodies of water using a global constant cutoff value of 0.2. Any part of the world that has an altitude of less than this cutoff would be filled with water. This approach was later modified because it would allow water to spawn in deserts. The new approach is to model the water level as an environment map, which sets the cutoff value appropriately lower for regions with higher temperatures. With this new temperature-sensitive calculation of the water level, a large region of high temperature can be completely devoid of water sources.

Creating rivers presents the challenge of having to be continuous. This is particularly difficult in a chunk-based, locally calculated system architecture like the one used in this project. As a noise map $\mathcal{N}$ is continuous, for any two points p_1 and p_2 with noise values $\mathcal{N}(p_1) > 0.5$ and $\mathcal{N}(p_2) < 0.5$, there has to exist a point somewhere between p_1 and p_2 with a noise value of exactly 0.5. This divides the plane into regions with a noise value > 0.5 and regions with a value < 0.5 , with a fine continuous dividing line between the two types of regions at 0.5. This is visualized in Figure 4. When a river is placed at any coordinate with a noise value close to 0.5, continuous rivers emerge with no start or end point.

The river generation uses this observation and applies a reshaping function shown in figure 5 to the river noise. The function maps values close to 0.5 to 0.8, and other values remain 0. This river map can then be subtracted from the terrain map to carve trenches. In most regions, these trenches will be naturally filled with water due to the water cutoff. In regions with high temperatures, the trenches are not filled with water because the water level there is low.

D. Atmospheric and Climate Simulation

The Atmospheric and Climate Simulation engine consists of three interdependent systems: a thermodynamic model, a

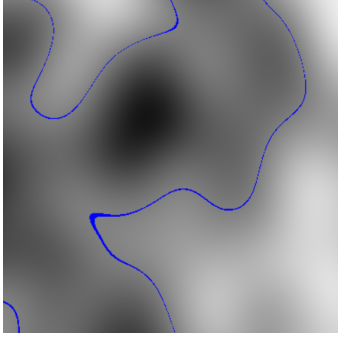


Fig. 4: Rivers form directly from noise by selecting values of 0.5 and coloring them blue

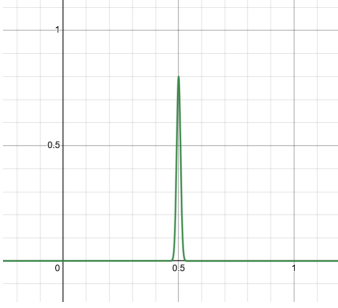


Fig. 5: River reshaping function drawn in Desmos

wind field simulation, and a reverse-tracing algorithm. These components operate sequentially to simulate realistic weather patterns within an infinite world.

Unlike scalar fields such as altitude or temperature, the wind is modeled as a continuous vector field $\vec{W}(x, y)$. To generate non-uniform, natural wind currents, two independent layers of Noise are utilized, representing the zonal (u) and meridional (v) components of the wind vector:

$$\vec{W}(x, y) = \langle \mathcal{N}_u(x, y), \mathcal{N}_v(x, y) \rangle \quad (2)$$

where $\mathcal{N}$ represents a simplex noise generator with frequency $f \propto 1/\text{scale}$. By combining these components, the system produces a coherent direction and magnitude for every coordinate in the infinite world, preventing a static "prevailing wind".

To achieve realistic hydrology, the movement of moisture over time is simulated using a Reverse Characteristic Tracing algorithm.

Although the climate pipeline that this project employs is based on the AutoBiomes framework, there is a fundamental limitation in their forward-propagation approach when applied to an infinite environment. In a finite map, moisture can be simulated by moving air masses forward from a known boundary. However, in an infinite procedurally generated world, this is mathematically impossible, as there is no defined "source" boundary; the wind originates from an infinitely distant coordinate. To resolve this, the simulation direction can be inverted, implementing a Reverse Characteristic Tracing algorithm. Instead of simulating fluid dynamics forward, the

moisture in a target pixel P_0 is calculated by following a "virtual air parcel" backward against the wind vector $\vec{W}$. This guarantees that the simulation is always local and bounded.

The trace follows the path defined iteratively for a finite number of steps K , at each step k , the air parcel interacts with the terrain and water bodies to update its carried moisture. The update logic consists of three physical interactions:

- **Moisture Uptake (Evaporation):** If the parcel travels over the ocean ($H < H_{sea}$) or a generated river, it accumulates moisture based on surface proximity ($M \leftarrow M + \delta_{pickup}$). This allows large river systems to act as secondary moisture sources for downwind terrain.
- **Continental Decay:** As the parcel travels over dry land, it slowly loses moisture ($M \leftarrow M - \delta_{decay}$), simulating the natural dissipation of clouds as they move inland.
- **Orographic Rain Shadow:** If the parcel encounters a mountain obstacle ($H > 0.6$), it is forced upward and loses a significant portion of its moisture ($M \leftarrow M - \delta_{mountain} \cdot (H - 0.6)$). This simulates the *Rain Shadow* effect, creating arid deserts on the leeward side of peaks.

After the wind simulation, a final pass applies a "Groundwater" override. If a coordinate is physically close to a riverbed or the coastline, its moisture value is boosted. This ensures that vegetation can thrive along riverbanks (forming riparian zones) even in arid environments where atmospheric rainfall is zero (e.g., an oasis or the Nile delta).

This tracing process, even if accurate, presents a significant performance bottleneck: tracing wind vectors for 80 steps for every single pixel in a 32×32 chunk requires a lot of computational power. To address this, this paper implements a low-resolution calculation strategy. The moisture values are calculated on a reduced 8×8 grid, drastically reducing the simulation count. After that, Bilinear Interpolation is used to upscale these simplified data back to the full 32×32 resolution. This optimization results in a 16x performance gain, allowing for fluid real-time generation without perceptible loss in visual quality.

E. Biome generation

The Biome generation is done using a Whittaker diagram, which classifies the biomes by temperature and precipitation. This rule-based approach creates believable results, relies only on local data, and a precalculated, discretized lookup table $Temperature \times Precipitation \rightarrow Biome$ achieves good performance. After initially mapping the biomes according to the Whittaker diagram, the biomes are blended at the edges. This can be done efficiently with Python's `scipy.ndimage` library.

Unlike all other environment maps, the biome map does not assign a single value to each coordinate, but a vector $[0, 1]^N$ of dimension N where N is the number of biomes. This vector represents a probability vector, and all its entries sum up to 1. In this way, every coordinate gets multiple biomes at once with different weights, and the strict and unnatural biome classification is replaced by a smooth transition.

F. Object Placement

The final stage of the pipeline transitions from continuous environmental fields (terrain, temperature, moisture) to discrete asset generation. This layer is responsible for populating the world with trees, rocks, and vegetation in a way that respects both the physical biome rules and the requirement for deterministic reproducibility.

A common challenge in infinite world generation is the "Order Dependency" problem. Standard pseudo-random number generators rely on a global state; if a player visits Chunk A, then Chunk B, and later returns to Chunk A, the generation order changes, causing assets to shift or disappear.

To ensure that a specific tree always appears at the same coordinate regardless of the player's path, this project discards the global random state in favor of Spatial Hashing.

For every chunk at coordinates (C_x, C_y) , a unique and deterministic seed is calculated using bitwise operations:

$$Seed_{chunk} = Seed_{global} \oplus (C_x \times P_1) \oplus (C_y \times P_2) \quad (3)$$

where P_1 and P_2 are large prime constants. This seed initializes a local random number generator, which produces a 32×32 "Chance Map" (a grid of white noise values $\in [0, 1]$). This mechanism ensures that the world remains consistent across multiple generations.

Once the deterministic framework is established, the placement logic is determined by the Biome Probability Map. Rather than iterating through coordinates, a vectorized masking is utilized to generate a "Density Mask" for each asset type.

The system applies specific density rules to the probability vector $\vec{v}$:

- **Forests:** These biomes have the highest asset density. They contain oak trees, palms, or snow-covered pines, depending on the specific subtype. Additional ground cover includes mushrooms and berry bushes in wetter variations.
- **Deserts:** Arid regions utilize a lower density mask. Cacti, dead bushes, and small rocks are placed.
- **Tundra and Alpine:** To simulate harsh and exposed environments, these biomes are free of vegetation. Instead, they feature a high density of geological features (rocks).
- **Grasslands:** To differentiate plains from empty terrain, a medium-density mask of wildflowers is applied, creating visual interest.

Since asset placement inherits its continuity from the blended biome map, a forest does not end abruptly. Instead, it fades out into neighboring grasslands, and the trees are placed more and more sparse, the lower the biome value for "forest" becomes.

VI. RESULTS

To validate the effectiveness of the physically-based generation pipeline, the system's output was analyzed across three key metrics: hydrological consistency, ecological coherence, and computational performance.

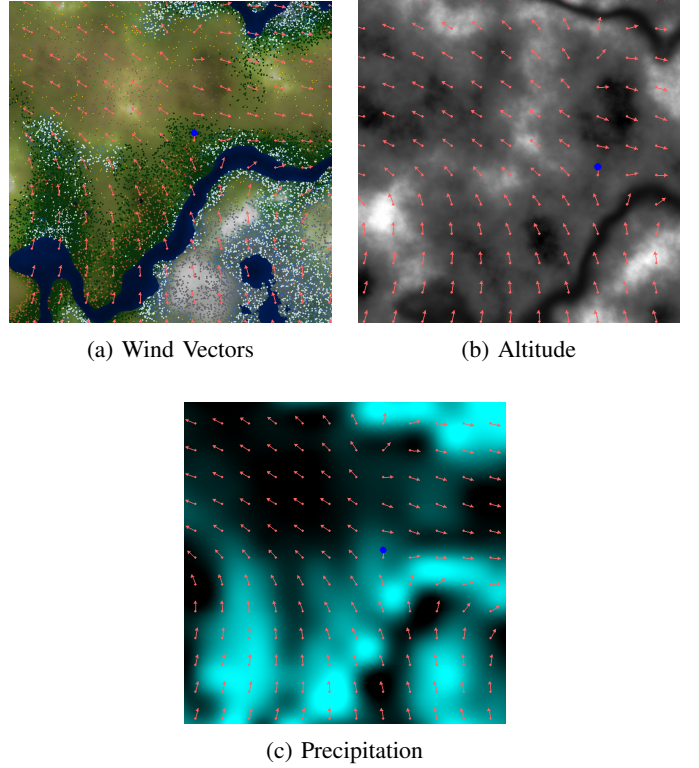


Fig. 6: Visualizing the Orographic Effect. (a): Wind flows West-to-East. (b): The Altitude Map showing the mountain ridge. (c): The resulting Precipitation Map shows the distinct rain shadow.

A. Simulation of Orographic Climate

The core hypothesis of this work was that simulating wind transport would result in a more realistic moisture distribution than simple noise maps. Figure 6 confirms this behavior.

As illustrated, the system successfully reproduces the "Rain Shadow" effect. The mountain range acts as a moisture barrier, preventing rainfall from reaching the leeward side. This behavior emerges entirely from the interaction between the terrain and wind vector fields, without explicit hard-coding of desert locations.

B. Ecological Transitions

The probabilistic Whittaker classification system produces smooth ecotones rather than abrupt biome changes. Figure 7 demonstrates a transition zone where moisture gradually increases due to ocean proximity. The terrain shifts organically from Desert to Grassland, eventually reaching a Tropical Forest. The continuous blending of textures at the boundaries eliminates the "patchwork" artifact common in discrete Voronoi-based biome systems.

C. Asset Distribution

Beyond global biomes, the system successfully generates localized ecological features based on specific rules. Figure 8 illustrates four distinct biomes: a Desert with sparse cacti,

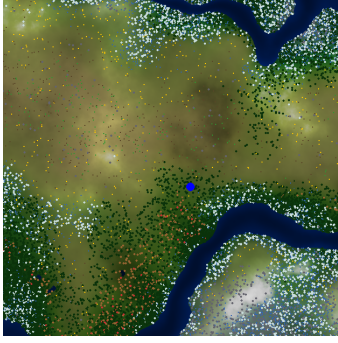
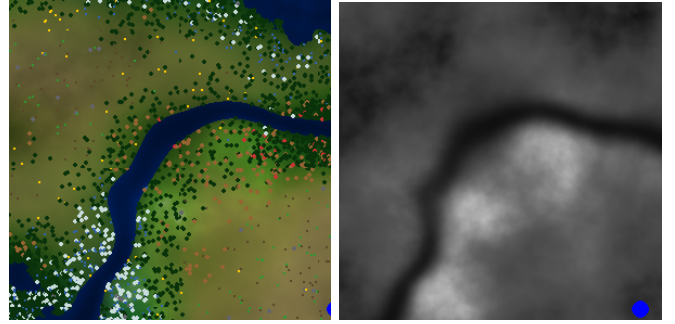


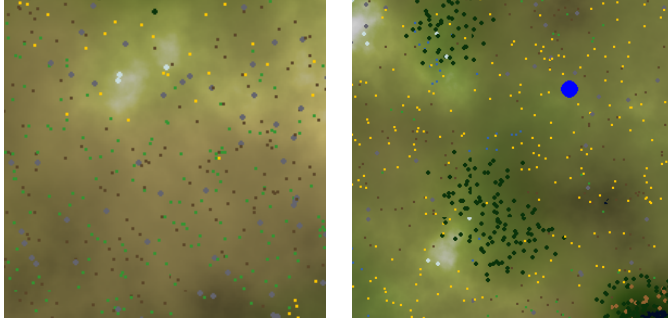
Fig. 7: Visualizing the transition between biomes



(a) River

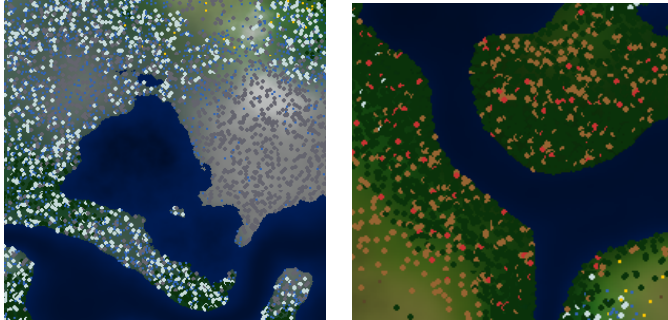
(b) River Altitude

Fig. 9: Visualizing the Rivers formation



(a) Desert

(b) Grassland



(c) Tundra and Taiga

(d) Rain Forest

Fig. 8: Visualizing the biomes object distribution

dead bushes, and rocks; a Grassland populated with yellow flowers and scattered trees; a combined Taiga and Tundra, where the Taiga features snow-covered trees and berry bushes, while the Tundra contains only rocks; and finally, a Tropical Rainforest characterized by a dense concentration of trees, including tropical varieties, as well as mushrooms.

D. Hydrological Networks

Unlike standard noise-based generation, which often results in disconnected "puddles," the river engine successfully generates coherent water channels. Figure 9 illustrates a river generated within a Subtropical Desert biome.

The visual results demonstrate that the engine accurately identifies river paths and lowers the altitude $H(x, y)$ of the terrain to form realistic trenches, ensuring that the water appears at the correct sea level. Additionally, the simulation

shows that rivers can override local biome rules: even when global wind patterns designate a chunk as "Desert," the areas surrounding the river are classified as "Grassland" or "Forest," effectively creating an oasis ecosystem.

E. Performance Analysis

A critical requirement for an infinite world is real-time performance. The generation time of a single chunk using the naive pixel-perfect wind trace (32×32 resolution) was compared against the multi-resolution optimization (8×8 grid with bilinear upscaling).

Method	Traces per Chunk	Avg. Time (ms)
Naive Tracing	1024	631.37 ms
Optimized (Upscaling)	64	47.84 ms

TABLE I: Impact of Multi-Resolution Optimization on Generation Time

As shown in Table I, the optimization results in an approximate 13.20x performance improvement.

VII. FUTURE WORK

Right now, the system uses vector noise to create global wind patterns. In the future, wind vectors could depend directly on temperature. In real-world physics, air moves from cooler, high-pressure areas to warmer, low-pressure ones. By calculating changes in the temperature map, wind patterns could form naturally based on heat instead of relying on random noise. This would also make it possible to simulate more complex effects, such as seasonal monsoons or local sea breezes. Additionally, a "top-down" method is currently being used for rivers: a noise map decides where rivers go, and the terrain is adjusted to fit them. A better approach would be a "bottom-up" hydraulic erosion model. Here, moisture falls on the terrain, gathers in low spots, and flows downhill using the height map. This would let rivers form naturally in valleys, create lakes, and merge into branching river systems, avoiding unnatural or illogical river paths. Moreover, there are other concepts that could also be implemented as an improvement of this project. Instead of a 2D approach, a 3D World infinite framework could provide new insights into the results.

VIII. CONCLUSION

This project successfully demonstrates that coherent infinite 2D top-down landscapes can be effectively generated by layering local physical simulations over continuous noise fields. The development of a Reverse Characteristic Tracing algorithm to simulate moisture in an unbounded environment was a fundamental finding, which successfully reproduced natural phenomena like the rain shadow effect without global context. Moreover, the implementation of multi-resolution upscaling confirmed that these complex climatic interactions remain computationally viable for real-time execution. Ultimately, the results validate that prioritizing local consistency through rule-based systems provides a scalable and robust foundation for procedural game environments.

ACKNOWLEDGMENTS

We would like to express our gratitude to Professor Jakub Kowalski for his continuous support during the course "Artificial Intelligence <3 Games: Procedural Content Generation" at the University of Wrocław. His mentorship was essential in shaping the project's initial concepts and improving our existing development with fresh insights. We also thank him for presenting us with this publication opportunity and for his guidance throughout the process.

REFERENCES

- [1] A. Khalifa, M. C. Green, G. A. B. Barros, and J. Togelius, "Intentional computational level design," *CoRR*, vol. abs/1904.08972, 2019. [Online]. Available: <http://arxiv.org/abs/1904.08972>
- [2] M. Janovský, P. Tobias, and V. Cehák, "3d visualisation of the historic predam vltava river valley—procedural and cad modelling, online publishing and virtual reality," *ISPRS International Journal of Geo-Information*, vol. 11, no. 7, 2022.
- [3] G. Alomia, X. Luo, C. Zúñiga, A. N. Cadavid, and C. Lozano-Garzón, "Automatic 3d urban installation generation in virtual cities," in *2019 International Conference on Virtual Reality and Visualization (ICVRV)*, 2019, pp. 217–222.
- [4] R. Fischer, P. Dittmann, R. Weller, and G. Zachmann, "Autobiomes: procedural generation of multi-biome landscapes," *Vis. Comput.*, vol. 36, no. 10–12, p. 2263–2272, Oct. 2020. [Online]. Available: <https://doi.org/10.1007/s00371-020-01920-7>